Pricing an option on many cores

SIMD, OpenMP and hybrid MPI for a Heston PDE solver

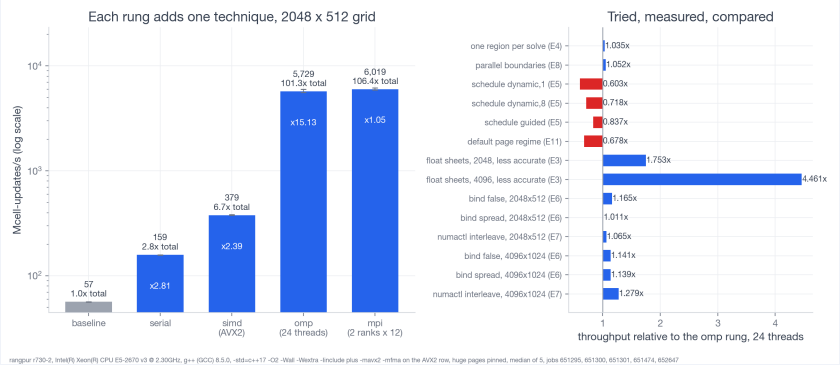
Zain Al-Saffi, 48008361

COSC3500 Milestone 2

October 2026

101 times faster on one node

Every version gives the same answer to 15 digits



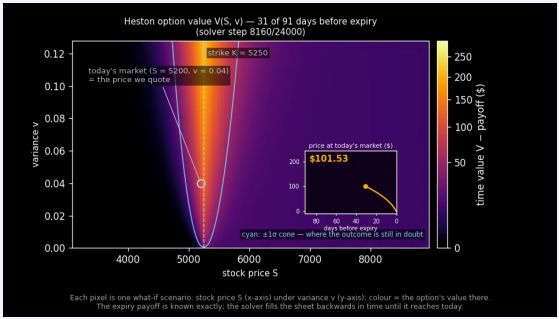
- Four techniques, each measured on its own, from 57 to 5729 million cell updates per second.

The problem

The price today of the right to buy at a fixed strike



At expiry the value is the payoff.



Mid-solve. Each step walks one step back towards today.

- Heston turns the price into a PDE in stock price, variance and time. The same formula runs on a million cells every step.

The data structure

Two flat arrays of doubles, read one, write the other, swap

the grid, two flat sheets include/grid.h:36–43

```
    // Position of node (stock_i, var_j) in the flat buffer.
    // Each row is laid out one after another in memory.
    int index(int stock_i, int var_j) const { return var_j * num_stock_nodes_ + stock_i; }

    // The sheet being read this step, given back read only.
    const std::vector<double>& current() const { return current_; }
    // The sheet being written this step.
    std::vector<double>& next() { return next_; }
```

the loop nest src/solver_baseline.cpp:42–63

```
    for (int step = 1; step <= num_timesteps; ++step) {
        // auto lets the compiler work out the long clock type for us
        auto step_start = std::chrono::steady_clock::now();

        // Every interior cell gets the full PDE
        for (int var_j = 1; var_j < grid.num_variance_nodes() - 1; ++var_j) {
            for (int stock_i = 1; stock_i < grid.num_stock_nodes() - 1; ++stock_i) {
                // Stock price and variance at this cell
                double stock_price = grid.stock_price(stock_i);
                double variance = grid.variance(var_j);

                // This cell and its eight neighbours.
                // East and west are one stock node away, north and south one variance row away
                double centre = current[grid.index(stock_i, var_j)];
                double east = current[grid.index(stock_i + 1, var_j)];
                double west = current[grid.index(stock_i - 1, var_j)];
                double north = current[grid.index(stock_i, var_j + 1)];
                double south = current[grid.index(stock_i, var_j - 1)];
                double north_east = current[grid.index(stock_i + 1, var_j + 1)];
                double north_west = current[grid.index(stock_i - 1, var_j + 1)];
                double south_east = current[grid.index(stock_i + 1, var_j - 1)];
                double south_west = current[grid.index(stock_i - 1, var_j - 1)];
```

Each sheet is ns x nv doubles stored row after row. East and west are next door in memory, north and south are one row, ns doubles, away. At 2048 x 512 a row is 16 KiB, a sheet 8 MiB, and the two sheets 16 MiB.

16 KiB

one row of 2048 doubles

8 MiB

one sheet at 2048 x 512

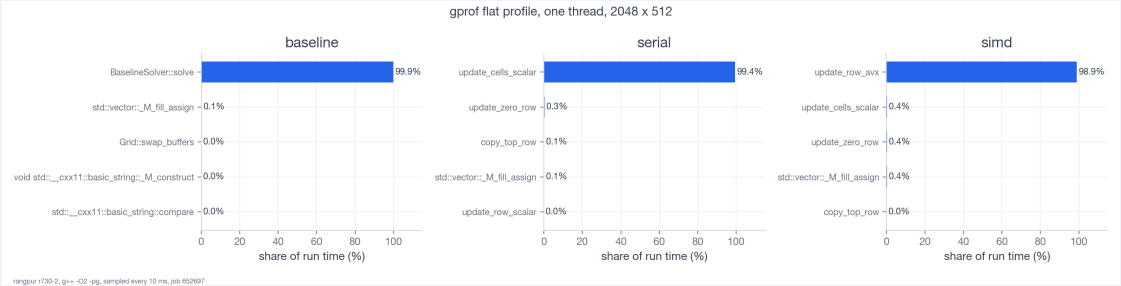
16 MiB

both sheets

- The stock axis is contiguous, so the inner loop walks memory in order.

Where the time goes

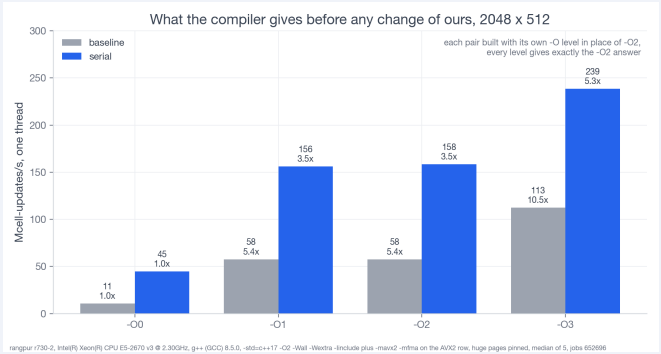
gprof flat profile, one thread, 2048 x 512



- The interior row update holds 99 percent of the time. The boundaries are under half a percent on one thread.

What the compiler does alone

The baseline at -O0 to -O3, same answer at every level



- 11 at -O0, 58 at -O2. -O3 doubles it by vectorising with two-lane SSE.

The serial rung

Divisions replaced by multiplications, once per solve

serial, one cell per pass src/stencil.cpp:105–121

```
        // Central differences give each derivative of the option value V
        double dV_dS = (east - west) * inv_two_ds;
        double d2V_dS2 = (east - 2.0 * centre + west) * inv_ds_sq;
        double dV_dv = (north - south) * inv_two_dv;
        double d2V_dv2 = (north - 2.0 * centre + south) * inv_dv_sq;
        double d2V_dS_dv = (north_east - north_west - south_east + south_west) * inv_four_ds_dv;

        // The five terms of the Heston PDE
        double stock_diffusion_term = stock_diffusion * stock_price_sq[stock_i] * d2V_dS2;
        double cross_term = cross_diffusion * stock_price[stock_i] * d2V_dS_dv;
        double variance_diffusion_term = variance_diffusion * d2V_dv2;
        double stock_drift_term = stock_drift * stock_price[stock_i] * dV_dS;
        double variance_drift_term = variance_drift * dV_dv;

        // Step back one dt from the current value
        double change = stock_diffusion_term + cross_term + variance_diffusion_term + stock_drift_term + variance_drift_term - rate * centre;
        out_row[stock_i] = centre + dt * change;
```

One cell per pass. Five derivatives, five PDE terms, one step back in time. Measured 159 Mcell-updates/s, 2.8x the baseline.

- The compiler may not do this, because it changes the last bits.

159 million updates a second, **2.8x** the baseline.

SIMD with AVX2

Four neighbouring cells per instruction, fused multiply-add

serial, one cell src/stencil.cpp:112–121

```
        // The five terms of the Heston PDE
        double stock_diffusion_term = stock_diffusion * stock_price_sq[stock_i] * d2V_dS2;
        double cross_term = cross_diffusion * stock_price[stock_i] * d2V_dS_dv;
        double variance_diffusion_term = variance_diffusion * d2V_dv2;
        double stock_drift_term = stock_drift * stock_price[stock_i] * dV_dS;
        double variance_drift_term = variance_drift * dV_dv;

        // Step back one dt from the current value
        double change = stock_diffusion_term + cross_term + variance_diffusion_term + stock_drift_term + variance_drift_term − rate * centre;
        out_row[stock_i] = centre + dt * change;
```

simd, four cells src/stencil_avx.cpp:99–110

```
        // Add up the five PDE terms and the discounting.
        // fmadd(a, b, c) works out a times b plus c with only one rounding
        __m256d change = _mm256_mul_pd(stock_diffusion_factor, d2V_dS2);
        change = _mm256_fmadd_pd(cross_factor, d2V_dS_dv, change);
        change = _mm256_fmadd_pd(variance_diffusion, d2V_dv2, change);
        change = _mm256_fmadd_pd(stock_drift_factor, dV_dS, change);
        change = _mm256_fmadd_pd(variance_drift, dV_dv, change);
        change = _mm256_fnmadd_pd(rate, centre, change);

        // Step back one dt and store the four new values
        __m256d new_value = _mm256_fmadd_pd(dt, change, centre);
        _mm256_storeu_pd(out_row + stock_i, new_value);
```

The same sum on four neighbouring cells at once, each multiply and add fused into one instruction. Measured 2.4x over serial.

96%

of scalar peak before SIMD

1.5x

compiler at -O3 with two lanes

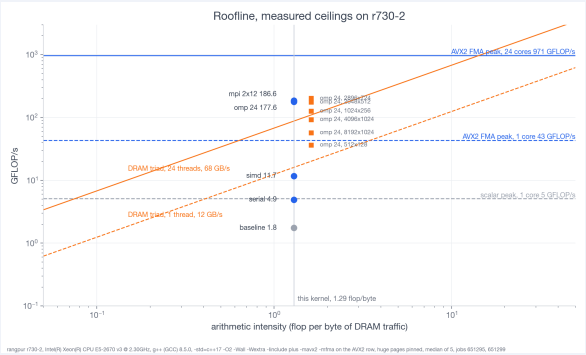
2.4x

by hand with four lanes and FMA

- 379 million updates a second. Not 4x, because each group of four loads nine overlapping windows.

SIMD on the roofline

Measured ceilings on one r730 node



- The SIMD rung does 12 GFLOP/s against an AVX2 ceiling of 43 for one core.

OpenMP

One pragma shares the rows of each step between threads

simd, one thread src/solver_simd.cpp:45–48

```
        // Every interior row, four cells at a time within each row
        for (int var_j = 1; var_j < num_variance_nodes - 1; ++var_j) {
            update_row_avx(constants, current, next, var_j);
        }
```

omp, every core src/solver_omp.cpp:44–49

```
        // Split the interior rows between the threads, one equal band each.
        // Threads only read current and only write their own rows of next, so no locks are needed
        #pragma omp parallel for schedule(static) default(none) shared(constants, current, next, num_variance_nodes)
        for (int var_j = 1; var_j < num_variance_nodes - 1; ++var_j) {
            update_row_avx(constants, current, next, var_j);
        }
```

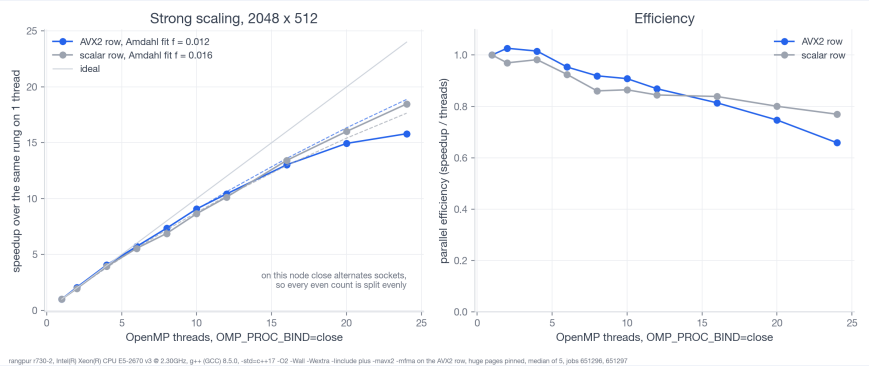
One line splits the rows between 24 threads. Each thread writes only its own rows, so no locks. Measured 15.1x over simd.

- Each thread reads only the old sheet and writes only its own rows.
- So there are no locks, and the answer equals one thread bit for bit.

- **15.8x** on 24 cores.

Strong scaling

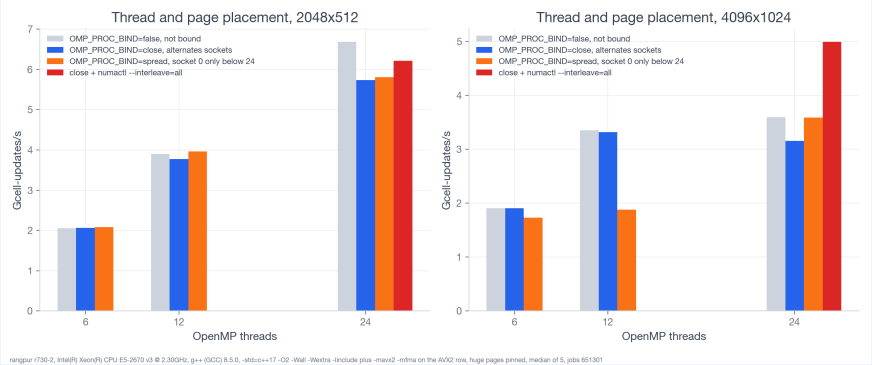
Fixed 2048 x 512 grid, 1 to 24 threads



- Amdahl fit, serial fraction 1.2 percent. The scalar row scales better, 18.5x, because it moves data at half the rate.

Placement

Which socket the threads and the pages sit on



- 12 threads on one socket 1880, over both sockets 3319.

Interleaving the pages gave the big grid 28 percent more.

Hybrid MPI

A slab of rows per process, one halo row swapped with each neighbour

mpi, swapping edge rows each step src/solver_mpi.cpp:56–80

```
    // The bottom process has nobody below it
    if (rank > 0) {
        // Post the receive first so the message can land straight in place
        MPI_Request receive_below;
        MPI_Irecv(halo_below, num_stock_nodes, MPI_DOUBLE, rank - 1, halo_tag, MPI_COMM_WORLD, &receive_below);
        requests.push_back(receive_below);

        MPI_Request send_below;
        MPI_Isend(first_row, num_stock_nodes, MPI_DOUBLE, rank - 1, halo_tag, MPI_COMM_WORLD, &send_below);
        requests.push_back(send_below);
    }

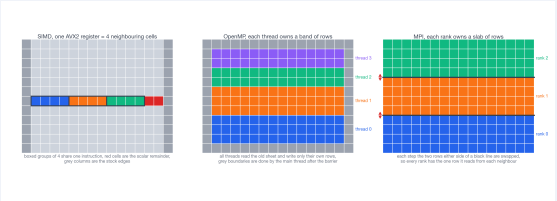
    // The top process has nobody above it
    if (rank < num_ranks - 1) {
        MPI_Request receive_above;
        MPI_Irecv(halo_above, num_stock_nodes, MPI_DOUBLE, rank + 1, halo_tag, MPI_COMM_WORLD, &receive_above);
        requests.push_back(receive_above);

        MPI_Request send_above;
        MPI_Isend(last_row, num_stock_nodes, MPI_DOUBLE, rank + 1, halo_tag, MPI_COMM_WORLD, &send_above);
        requests.push_back(send_above);
    }

    // Nothing may touch these rows until every send and receive has finished
    MPI_Waitall((int)requests.size(), requests.data(), MPI_STATUSES_IGNORE);
```

Each process owns a slab of rows and swaps one edge row with each neighbour every step. On one node 2x12 beat 1x24 by 1.05x.

Receives are posted before sends.

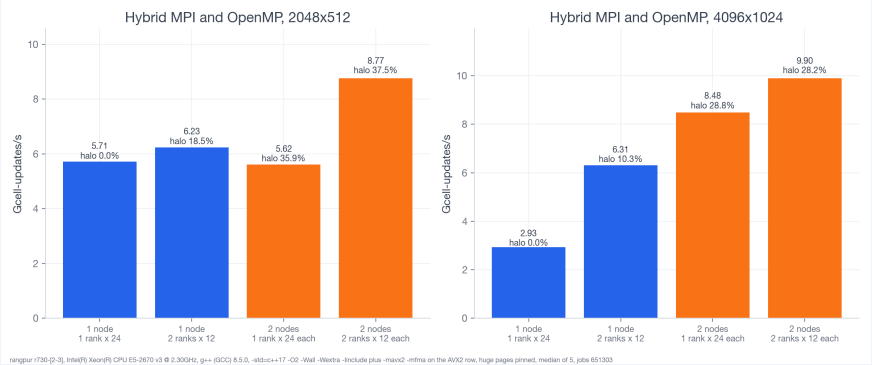


Which cells each lane, thread and rank owns.

- One process of 12 threads per socket first-touches its own copy of the grid on its own socket.

Hybrid layouts

One and two nodes, two grid sizes



- Two nodes give **1.41x** one node, 70 percent efficient. Each halo exchange costs about 45 microseconds.

Same answer on every rung

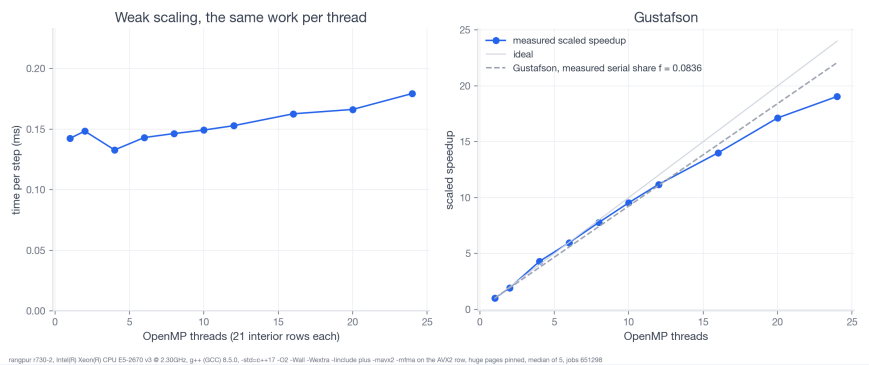
Price, delta, gamma and vega against the baseline



- Every rung within 3.3e-15. OpenMP and MPI equal the SIMD rung exactly, and every job reran the tests before timing.

Weak scaling

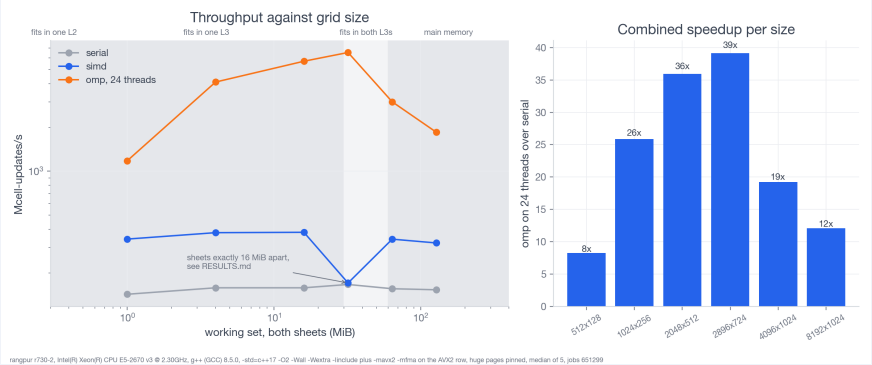
21 rows per thread, the grid grows with the threads



- Step time 0.14 to 0.18 ms, a scaled speedup of 19 on 24 threads.

Grid size

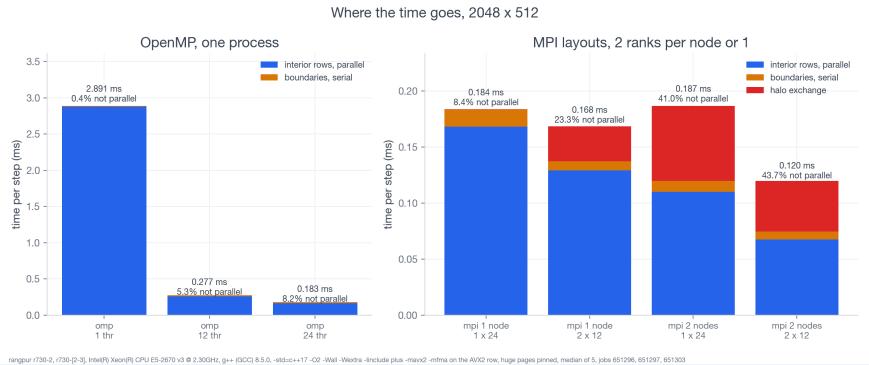
1 MiB to 128 MiB of working set



- The 2048 grid fits in the two L3 caches and runs 36x serial. At 4096 by 1024 it spills to memory and the rate halves.

Where each step goes

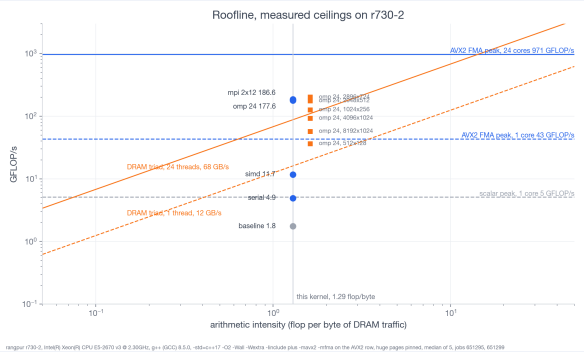
Parallel rows, serial boundaries and halo exchange



- At 24 threads the serial boundaries are 8 percent of the step, not the half percent I expected.

After threads, bandwidth

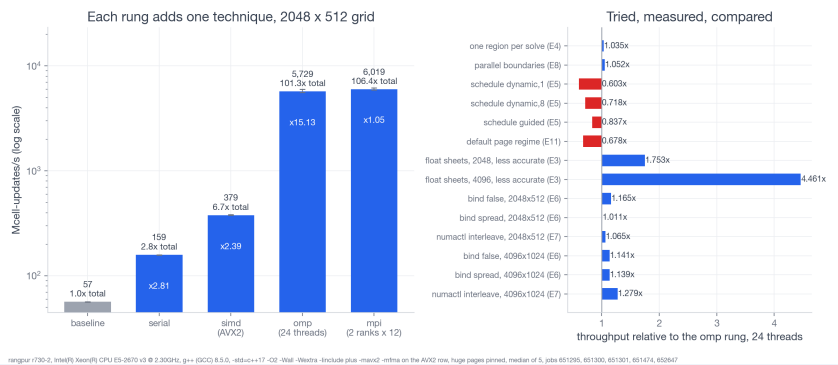
The roofline again



- The large grid sits on the memory roof, so the limit is bandwidth, not arithmetic.

Tried and rejected

Each bar is a variant against the OpenMP rung



- Float was 1.75x faster and 4.5x on the big grid, but moved the price by 4 cents and gamma by 12 percent.

Conclusion

The ladder against the measured ceilings

Rung	Mcell/s	Total
baseline	57	1x
serial	159	2.8x
simd	379	6.7x
omp, 24 threads	5729	101x
mpi, two nodes	8769	**155x**

Ceiling	Measured
scalar peak, one core	5.1 GFLOP/s
AVX2 peak, one core	43 GFLOP/s
DRAM triad, 24 threads	68 GB/s

- Serial sat at the scalar peak, so SIMD was the only way up.
- Threads took the large grid onto the memory roof.

- Four of six hypotheses held. The serial boundary and the network did not. Next is overlapping the halo with the interior rows.

101x on one node, 155x on two, the same answer on every rung

Every number has a job id in docs/RESULTS.md

COSC3500 Milestone 2, Zain Al-Saffi